포팅 매뉴얼

생성일	@2025년 11월 19일 오전 9:37

1️⃣ 배포 이미지
- ✅ 1.1 AI
  - ✔️ 1.1.1 LLM Dockerfile
  - ✔️ 1.1.2 Judge LLM Dockerfile
  - ✔️ 1.1.3 Judge Prompt Dockerfile
  - ✔️ 1.1.4 Judge Prompt model_server
  - ✔️ 1.1.5 Judge LLM model_server
- ✅ 1.2 Backend
  - ✔️ 1.2.1 Backend
- ✅ 1.3 Frontend
  - ✔️ 1.3.1 Frontend Dockerfile
  - ✔️ 1.3.2 nginx.conf

2️⃣ Infra 설정
- ✅ 2.1 Main Server
  - ✔️ 2.1.1 디렉토리 구조
  - ✔️ 2.1.2 Docker compose
  - ✔️ 2.1.3 .env(main llm)
- ✅ 2.2 Judge Prompt Server
  - ✔️ 2.2.1 Docker compose
  - ✔️ 2.2.2 .env(jp_server)
  - ✔️ 2.2.3 .env(model_server)
- ✅ 2.3 Judge LLM Server
  - ✔️ 2.3.1 Docker compose
  - ✔️ 2.3.2 .env(api)
  - ✔️ 2.3.2 .env(model)
- ✅ 2.4 Docker 설치
  - ✔️ 2.4.1 AMD 버전
  - ✔️ 2.4.2 ARM 버전
- ✅ 2.5 Gitlab
  - ✔️ 2.5.1 Gitlab Runner 서버
  - ✔️ 2.5.2 .gitlab-ci.yml

3️⃣ Jenkins 설정
- ✅ 3.1 Dockerfile
- ✅ 3.2 Jenkins pipeline settings
- ✅ 3.3 Jenkins Credentials
- ✅ 3.4 Jenkins Script

4️⃣ 개발 환경
- ✅ 4.1 FrontEnd

✔️ 4.1.1 주요 프레임워크 및 라이브러리
✔️ 4.1.2 데이터 통신 및 실시간
✔️ 4.1.3 개발 도구 및 환경 설정
✅ 4.2 BackEnd
✔️ 4.2.1 개발 환경
✔️ 4.2.2 언어 및 핵심 프레임워크
✔️ 4.2.3 데이터베이스 및 데이터 처리
✔️ 4.2.4 API 문서화
✅ 4.3 AI
✔️ 4.3.1 언어 및 프레임워크
✔️ 4.3.2 주요 라이브러리
레포지토리 구조

1️⃣ 배포 이미지

✅ 1.1 AI

✔️ 1.1.1 LLM Dockerfile

```
FROM python:3.12.3-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# 작업 디렉토리 설정
WORKDIR /app

# 필수 패키지 설치
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    git \
    fonts-nanum \
    fontconfig \
    && fc-cache -fv \
    && rm -rf /var/lib/apt/lists/*

RUN git clone https://github.com/EleutherAI/lm-evaluation-harness.git
RUN cd lm-evaluation-harness && pip install -e .

# requirements 파일 복사
COPY requirements.txt .
```

```
# python 패키지 설치
RUN pip install --no-cache-dir -r requirements.txt

# 애플리케이션 코드 복사
COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

✔️ 1.1.2 Judge LLM Dockerfile

```
FROM python:3.12.3-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app/judge_llm

# 의존성 설치 (빌드 캐시 최적화)
COPY api/requirements.txt /app/judge_llm/api/requirements.txt
RUN pip install --no-cache-dir -r /app/judge_llm/api/requirements.txt

# 앱 소스 복사 (루트 컨텍스트 전체 포함 -> main.py 포함)
COPY . /app/judge_llm

EXPOSE 8081

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8081"]
```

✔️ 1.1.3 Judge Prompt Dockerfile

```
# Python 3.12.3 베이스 이미지 사용
FROM python:3.12.3-slim

# 작업 디렉토리 설정
WORKDIR /app

# 시스템 패키지 업데이트 및 필수 패키지 설치
RUN apt-get update && apt-get install -y \
    gcc \
```

```
    g++ \
    cmake \
    make \
    git \
    ninja-build \
    && rm -rf /var/lib/apt/lists/*

# requirements 파일 복사
COPY requirements_JP.txt .

# Python 패키지 설치 (numpy 먼저 설치 후 나머지 패키지 설치)
RUN pip install --no-cache-dir numpy==2.3.4 && \
    CMAKE_ARGS="-DGGML_NATIVE=OFF" CMAKE_BUILD_PARALLEL_LEVEL=4 pip i
nstall --no-cache-dir -r requirements_JP.txt



ENV NLTK_DATA=/usr/local/nltk_data
RUN mkdir -p /usr/local/nltk_data && \
    python -m nltk.downloader -d /usr/local/nltk_data stopwords punkt



# 애플리케이션 코드 복사
COPY app/ ./app/

# 환경 변수 설정 (기본값)
ENV PYTHONPATH=/app

# 포트 노출
EXPOSE 8000

# 애플리케이션 실행
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

✔️ 1.1.4 Judge Prompt model_server

```
cd judge_prompt

source venv/bin/activate

cd model_server

# 사용 GPU에 따라 설정
CMAKE_ARGS="-DLLAMA_METAL=on" \
```

```
pip install -U "llama-cpp-python[server]" --no-cache-dir

pip install -r requirements_JP.txt

uvicorn app.main:app --host 0.0.0.0 --port 8081 --reload
```

✔️ 1.1.5 Judge LLM model_server

```
cd judge_llm

source venv/bin/activate

cd model

# 사용 GPU에 따라 설정
CMAKE_ARGS="-DLLAMA_METAL=on" \
pip install -U "llama-cpp-python[server]" --no-cache-dir

pip install -r requirements_model.txt

uvicorn app.main:app --host 127.0.0.1 --port 8090 --reload
```

✅ 1.2 Backend

✔️ 1.2.1 Backend

```
# ----------------------------------------------------
# 1) Build Stage: Liberica OpenJDK 17 (Full JDK)
# ----------------------------------------------------
FROM bellsoft/liberica-openjdk-debian:17 AS builder

WORKDIR /app
ENV TZ=Asia/Seoul LANG=C.UTF-8

# Gradle 캐시를 최대한 활용하기 위해 build 전에 의존성만 먼저 복사
COPY gradlew .
COPY gradle gradle
RUN chmod +x gradlew

# Gradle 설정 및 종속성 미리 다운로드
COPY build.gradle settings.gradle ./
```

```
RUN ./gradlew dependencies --no-daemon || true

# 소스 복사 후 빌드
COPY src ./src
RUN ./gradlew clean bootJar -x test --no-daemon

# jar 파일을 별도 폴더로 이동
RUN mkdir -p /out && cp build/libs/*.jar /out/app.jar


# ----------------------------------------------------
# 2) Runtime Stage: Liberica Runtime 17 (경량 JRE)
# ----------------------------------------------------
FROM bellsoft/liberica-openjre-alpine:17

WORKDIR /app
COPY --from=builder /out/app.jar /app/app.jar

ENV SERVER_PORT=8080
EXPOSE 8080


ENTRYPOINT ["java", "-jar", "/app/app.jar", "--server.port=${SERVER_PORT}"]
```

✅ 1.3 Frontend

✔️ 1.3.1 Frontend Dockerfile

```
# ============================
# 1️⃣ Build Stage
# ============================
FROM node:22-alpine AS builder
WORKDIR /app

ARG VITE_API_URL
ENV VITE_API_URL=${VITE_API_URL}

COPY package*.json ./
RUN if [ -f package-lock.json ]; then npm ci; else npm install; fi
COPY . .
RUN npm run build
```

```
# ============================
# 2️⃣ Nginx Stage
# ============================
FROM nginx:alpine

# 기본 conf 제거(볼륨 마운트로 우리의 conf를 사용할 것)
RUN rm -f /etc/nginx/conf.d/default.conf

# 빌드 결과 복사 (정적 서빙 루트)
COPY --from=builder /app/dist /usr/share/nginx/html

# 포그라운드 실행
EXPOSE 80 443
CMD ["nginx", "-g", "daemon off;"]
```

✔️ 1.3.2 nginx.conf

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /var/run/nginx.pid;

events {
  worker_connections 1024;
}

http {
  resolver 127.0.0.11 ipv6=off valid=30s;
  resolver_timeout 5s;

  include       /etc/nginx/mime.types;
  default_type  application/octet-stream;
  access_log /var/log/nginx/access.log;
  sendfile on; tcp_nopush on; tcp_nodelay on;
  keepalive_timeout 65;

  upstream back_upstream {
    zone back_zone 64k;          # ★ 공유메모리 영역 추가
    server back:8080 resolve;    # ★ 동적 해석
    keepalive 32;
```

```
  }

  # 80: ACME만 허용, 그 외 443 리다이렉트
  server {
    listen 80 default_server;
    server_name ecoprompt.duckdns.org;

    # ✅ ACME 전용 webroot를 /var/www/certbot 로 사용
    location ^~ /.well-known/acme-challenge/ {
      root /var/www/certbot;
      try_files $uri =404;
      default_type text/plain;
    }

    location / {
      return 301 https://$host$request_uri;
    }
  }

  server {
    listen 443 ssl;
    http2  on;
    server_name ecoprompt.duckdns.org;

    ssl_certificate     /etc/letsencrypt/live/ecoprompt.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/ecoprompt.duckdns.org/privkey.pem;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;

    # ============== API / Swagger / OAuth2 ==============
    location = /api/v1 {
      return 301 /api/v1/;
    }

    location ^~ /api/v1/ {
      proxy_pass http://back_upstream;
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto https;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
```

```
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 3600; proxy_send_timeout 3600;
    proxy_buffering off;
    client_max_body_size 10M;
    add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0" al
ways;
    add_header Pragma "no-cache" always;
    add_header Expires "0" always;
  }

  location ^~ /oauth2/ {
    proxy_pass http://back_upstream;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Forwarded-Host  $host;
    proxy_set_header X-Forwarded-Port  443;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
    proxy_read_timeout 3600; proxy_send_timeout 3600;
    proxy_buffering off;
  }

  location = /swagger-ui.html {
    proxy_pass http://back_upstream$request_uri;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0" al
ways;
    add_header Pragma "no-cache" always;
    add_header Expires "0" always;
  }

  location ^~ /swagger-ui/ {
    proxy_pass http://back_upstream;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
```

```
    add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0" al
ways;
    add_header Pragma "no-cache" always;
    add_header Expires "0" always;
  }

  location ^~ /v3/api-docs {
    proxy_pass http://back_upstream;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0" al
ways;
    add_header Pragma "no-cache" always;
    add_header Expires "0" always;
  }

  # ============== 정적(프론트) 직접 서빙 ==============
  root   /usr/share/nginx/html;
  index  index.html;

  # 정적 캐싱(아이콘/폰트 포함)
  location ~* \.(js|css|png|jpg|jpeg|gif|svg|ico|woff|woff2|ttf|eot)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    try_files $uri =404;
  }

  # SPA fallback
  location / {
    try_files $uri $uri/ /index.html;
  }

  # 건강검진용
  location = /nginx-ok {
    default_type text/plain;
    return 200 'ok';
  }
 }
}
```

2️⃣ Infra 설정

✅ 2.1 Main Server

✔️ 2.1.1 디렉토리 구조

```
💡 infra
├── certbot
│       └── www
├── docker-compose.yml
├── env.prod
├── front
│       └── nginx.conf
├── jenkins
│       └── Dockerfile
├── jenkins-data
├── letsencrypt
├── mongodb
│       ├── data
│       ├── init
│       └── logs
├── mysql
│       ├── conf.d
│       ├── data
│       └── logs
├── nginx
│       ├── conf.d
│       ├── html
│       │       └── index.html
│       ├── logs
│       │       ├── access.log
│       │       └── error.log
│       ├── nginx.conf
│       └── ssl
└── redis
        ├── data
        └── logs
```

✔️ 2.1.2 Docker compose

```
services:
  mysql:
    image: mysql:8.0.42
    container_name: mysql-db
    restart: unless-stopped
    env_file:
      - ./env.prod/.env.mysql
    ports:
      - "3306:3306"
    volumes:
      - ./mysql/data:/var/lib/mysql
      - ./mysql/conf.d:/etc/mysql/conf.d
      - ./mysql/logs:/var/log/mysql
    networks:
      - eco-network

  redis:
    image: redis:8.2.2
    container_name: redis-cache
    restart: unless-stopped
    env_file:
      - ./env.prod/.env.redis
    ports:
      - "127.0.0.1:16379:6379"
    volumes:
      - ./redis/data:/data
      - ./redis/logs:/var/log/redis
    command:
      - "redis-server"
      - "--appendonly"
      - "yes"
    networks:
      - eco-network

  mongodb:
    image: mongo:7.0
    container_name: mongodb-db
    restart: unless-stopped
    env_file:
      - ./env.prod/.env.mongo
    ports:
      - "27017:27017"
```

```
    volumes:
      - ./mongodb/data:/data/db
      - ./mongodb/logs:/var/log/mongodb
      - ./mongodb/init:/docker-entrypoint-initdb.d
    networks:
      - eco-network

  back:
    image: ecoprompt/back:latest
    restart: unless-stopped
    env_file:
      - ./env.prod/.env.be
    depends_on:
      - mysql
      - redis
      - mongodb
    volumes:
      - /home/a309/private_key:/private_key
      - ../logs/be:/app/logs/be
    networks:
      - eco-network

  front:
    image: ecoprompt/front:latest
    container_name: front
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    env_file:
      - ./env.prod/.env.front
    depends_on: [ back ]
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf
      - ./nginx/conf.d:/etc/nginx/conf.d
      - ./nginx/ssl:/etc/nginx/ssl
      - ./nginx/logs:/var/log/nginx
      - ./letsencrypt:/etc/letsencrypt:ro
      - ./certbot/www:/var/www/certbot
    networks: [ eco-network ]
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost/nginx-ok"]
      interval: 15s
```

```
    timeout: 3s
    retries: 10
    start_period: 20s

  llm:
    image: ecoprompt/llm:latest
    container_name: llm
    ports:
      - "8000:8000"
    env_file:
      - ./env.prod/.env.llm
    restart: unless-stopped
    volumes:
      - ~/llm/local-models/Llama-SSAFY-8B:/app/local-models/Llama-SSAFY-8B
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: 1
              capabilities: [gpu]
    networks:
      - eco-network

  certbot:
    image: certbot/certbot:latest
    container_name: certbot
    volumes:
      - ./letsencrypt:/etc/letsencrypt             # ← 인증서 쓰기
      - ./nginx/html:/var/www/certbot              # ← 웹루트 챌린지 경로
    networks: [eco-network]

  jenkins:
    build: ./jenkins
    image: my-jenkins:lts-with-dockercli
    container_name: jenkins
    restart: unless-stopped
    ports:
      - "8080:8080"
    group_add:
      - "999"
    volumes:
      - ./jenkins-data:/var/jenkins_home
```

```
      - /var/run/docker.sock:/var/run/docker.sock

networks:
  eco-network:
    driver: bridge
```

✔️ 2.1.3 .env(main llm)

```
HUGGING_FACE_API_KEY=
MONGO_URL=mongodb://a309:a309@mongodb:27017

MAX_CONCURRENT_REQUESTS=5
REQUEST_TIMEOUT=300

TEAM_FOLDER_NAME=2ofsvz
BUCKET_NAME=ssafyapp-stg-project-prv-data
AWS_ACCESS_KEY=
AWS_SECRET_KEY=
```

✅ 2.2 Judge Prompt Server

✔️ 2.2.1 Docker compose

```
services:
  judge-prompt:
    image: ecoprompt/judge-prompt:latest
    container_name: judge-prompt
    ports:
      - "8013:8000"
    env_file:
      - .env
    volumes:
      - /Users/ssafy/S13P31A309/judge_prompt/models:/app/models
```

✔️ 2.2.2 .env(jp_server)

```
MODEL_PATH=./models/qwen2.5-7b/qwen2.5-7b-instruct-q5_k_m-00001-of-00002.
gguf
LLAMA_URL=http://host.docker.internal:8081
MODEL_NAME=qwen2.5-7b-instruct-gguf
```

```
QUEUE_MAXSIZE=128
WORKERS=4

CONNECT_TIMEOUT=2.0
READ_TIMEOUT=300.0
WRITE_TIMEOUT=35.0
POOL_TIMEOUT=5.0
RETRIES=1
CONCURRENCY_LIMIT=16
```

✔️ 2.2.3 .env(model_server)

```
# 모델 경로
MODEL_PATH=/Users/ssafy/S13P31A309/judge_prompt/models/qwen2.5-7b/qwen2.5-7b-instruct-q5_k_m-00001-of-00002.gguf
HOST=0.0.0.0
PORT=8081
N_CTX=16384
N_GPU_LAYERS=32
```

✅ 2.3 Judge LLM Server

✔️ 2.3.1 Docker compose

```
services:
  judge-llm:
    image: ecoprompt/juge-llm:latest
    container_name: judge-llm
    ports:
      - "8083:8081"
    env_file:
      - ./.env
    extra_hosts:
      - "host.docker.internal:host-gateway"
    restart: unless-stopped
```

✔️ 2.3.2 .env(api)

```
API_HOST=0.0.0.0
API_PORT=8081
```

```
# llama-server 연동
LLAMA_SERVER_URL=http://host.docker.internal:8090
# LLAMA_SERVER_URL=http://127.0.0.1:8090
LLAMA_SERVER_MODEL=/Users/ssafy/S13P31A309/judge_llm/model/prom2-q5_k_
m.gguf
JUDGE_ADAPTER=llama
JUDGE_TIMEOUT_S=40
JUDGE_TEXT_LIMIT=9999


# Main LLM 연동
MAIN_LLM_URL=
MAIN_LLM_TRAIN_PATH=/api/v1/ai/train
MAIN_LLM_TOKEN=
MAIN_LLM_TIMEOUT_S=30
MAIN_LLM_RETRIES=2

# MongoDB 연동
MONGO_URI=
ECO_MONGO_DB=
ECO_MASK_COLL=masking_message
ECO_MASK_UPSERT=true
MONGO_TIMEOUT_MS=5000

# 개발·디버그 평가 정책
JUDGE_TWO_PASS=true
JUDGE_MASK_PROMPT_FOR_EVAL=false
JUDGE_MASK_ANSWER_FOR_EVAL=false
JUDGE_USE_TOTAL=false
JUDGE_FINAL_THRESHOLD=3.5
JUDGE_DEBUG_RETURN_SUBSCORES=true

# 민감정보 마스킹 정책
ADDRESS_POLICY=CITY_GU

## 이메일
EMAIL_POLICY=KEEP_DOMAIN
DEBUG_MASK_PREVIEW=true
```

✔️ 2.3.2 .env(model)

```
LLAMA_SERVER_HOST=127.0.0.1
LLAMA_SERVER_PORT=8090
```

```
JUDGE_MODEL=$HOME/S13P31A309/judge_llm/model/prom2-q5_k_m.gguf
JUDGE_N_GPU_LAYERS=-1
JUDGE_N_THREADS=10
JUDGE_N_CTX=32768
JUDGE_N_BATCH=512
JUDGE_N_PARALLEL=2
LLAMA_CACHE_RAM=4096

JUDGE_MAX_TOKENS=384
JUDGE_TEMPERATURE=0
JUDGE_TIMEOUT_S=60
JUDGE_TEXT_LIMIT=9999
JUDGE_USE_JSON_FORMAT=true

WATCHDOG_INTERVAL_S=60
DO_LONG_WARMUP=1
```

✅ 2.4 Docker 설치

✔️ 2.4.1 AMD 버전

```
#!/bin/bash
set -e

echo "================================"
echo " 🔹 Step 1: 시스템 패키지 업데이트"
echo "================================"
sudo apt-get update -y

echo "================================"
echo " 🔹 Step 2: 필수 패키지 설치"
echo "================================"
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg-agent software-properties-common

echo "================================"
echo " 🔹 Step 3: Docker 공식 GPG 키 추가"
echo "================================"
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

echo "================================"
```

```
echo " 🔹 Step 4: Docker apt 저장소 추가"
echo "================================"
sudo add-apt-repository \
  "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable"

echo "================================"
echo " 🔹 Step 5: 저장소 업데이트"
echo "================================"
sudo apt-get update -y

echo "================================"
echo " 🔹 Step 6: Docker 설치"
echo "================================"
sudo apt-get install -y docker-ce docker-ce-cli containerd.io

echo "================================"
echo " 🔹 Step 7: Docker 서비스 상태 확인"
echo "================================"
sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker --no-pager

echo "================================"
echo " 🔹 Step 8: 현재 사용자(docker 그룹 추가)"
echo "================================"
sudo usermod -aG docker ubuntu

echo "================================"
echo "✅ Step 9: 그룹 변경 적용 (newgrp docker)"
echo "================================"
newgrp docker <<EONG
echo "✅ Docker 버전 확인:"
docker --version
echo "✅ Docker 테스트 실행:"
docker run --rm hello-world
EONG

echo "================================"
echo "🎉 Docker 설치 및 설정 완료!"
echo "================================"
```

✔️ 2.4.2 ARM 버전

```
#!/bin/bash
set -e

echo "================================"
echo " 🔹 Step 1: 시스템 패키지 업데이트"
echo "================================"
sudo apt-get update -y

echo "================================"
echo " 🔹 Step 2: 필수 패키지 설치"
echo "================================"
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg-agent software-properties-common

echo "================================"
echo " 🔹 Step 3: Docker 공식 GPG 키 추가"
echo "================================"
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -

echo "================================"
echo " 🔹 Step 4: Docker apt 저장소 추가 (ARM64용)"
echo "================================"
echo \
  "deb [arch=arm64] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list

echo "================================"
echo " 🔹 Step 5: 저장소 업데이트"
echo "================================"
sudo apt-get update -y

echo "================================"
echo " 🔹 Step 6: Docker 설치"
echo "================================"
sudo apt-get install -y docker-ce docker-ce-cli containerd.io

echo "================================"
echo " 🔹 Step 7: Docker 서비스 상태 확인"
echo "================================"
sudo systemctl enable docker
sudo systemctl start docker
sudo systemctl status docker --no-pager
```

```
echo "================================"
echo " 🔹 Step 8: 현재 사용자(docker 그룹 추가)"
echo "================================"
CURRENT_USER=$(whoami)
sudo usermod -aG docker $CURRENT_USER

echo "================================"
echo "✅ Step 9: 그룹 변경 적용 및 테스트"
echo "================================"
newgrp docker <<EONG
echo "✅ Docker 버전 확인:"
docker --version
echo "✅ Docker 테스트 실행:"
docker run --rm hello-world
EONG

echo "================================"
echo "🎉 Docker 설치 및 설정 완료! (ARM64 버전)"
echo "================================"
```

✅ 2.5 Gitlab

✔️ 2.5.1 Gitlab Runner 서버

```
# sudo cat /etc/gitlab-runner/config.toml

concurrent = 1
check_interval = 0
shutdown_timeout = 0

[session_server]
  session_timeout = 1800

[[runners]]
  name = "jenkins-runner"
  url = "https://lab.ssafy.com"
  id = 1504
  token = "GITLAB_RUNNER_TOKEN"
  token_obtained_at = 2025-10-28T01:48:49Z
  token_expires_at = 0001-01-01T00:00:00Z
  executor = "shell"
```

```
  run_untagged = true
  [runners.cache]
    MaxUploadedArchiveSize = 0
    [runners.cache.s3]
    [runners.cache.gcs]
    [runners.cache.azure]
```

참고

- https://docs.gitlab.com/runner/

✔️ 2.5.2 .gitlab-ci.yml

```
stages: [trigger_jenkins]

trigger:jenkins:
  stage: trigger_jenkins
  image: curlimages/curl:8.9.1
  rules:
    # 머지 후 dev/master로 "푸시"될 때만
    - if: '$CI_PIPELINE_SOURCE == "push" && ($CI_COMMIT_BRANCH == "dev" || $CI
_COMMIT_BRANCH == "master")'
      when: on_success
    - when: never
  script: |
    PREV_SHA=$(git rev-parse HEAD~1)
    echo "PREV_SHA=$PREV_SHA"
    CRUMB=$(curl -s -u "$JENKINS_USER:$JENKINS_API_TOKEN" "$JENKINS_URL/c
rumbIssuer/api/json" | jq -r '.crumb' )

    curl -X POST \
      -u "$JENKINS_USER:$JENKINS_API_TOKEN" \
      -H "Jenkins-Crumb: $CRUMB" \
      "$JENKINS_URL/job/$JENKINS_JOB_NAME/buildWithParameters" \
      --data-urlencode "token=$JENKINS_JOB_TOKEN" \
      --data-urlencode "BRANCH=$CI_COMMIT_BRANCH" \
      --data-urlencode "HEAD=$CI_COMMIT_SHA" \
      --data-urlencode "PREV=$PREV_SHA" \
      --data-urlencode "REPO_URL=https://lab.ssafy.com/s13-final/S13P31A309.git"
  variables:
    # GitLab → Jenkins 접속정보(프로젝트 CI/CD Variables에 정의해두기)
    JENKINS_URL: ${JENKINS_URL}
    JENKINS_JOB_NAME: ${JENKINS_JOB_NAME}
    # Jenkins > Job에서 설정한 trigger token
```

```
    JENKINS_JOB_TOKEN: ${JENKINS_JOB_TOKEN}
    # Jenkins 사용자/API 토큰
    JENKINS_USER: ${JENKINS_USER}
    JENKINS_API_TOKEN: ${JENKINS_API_TOKEN}
```

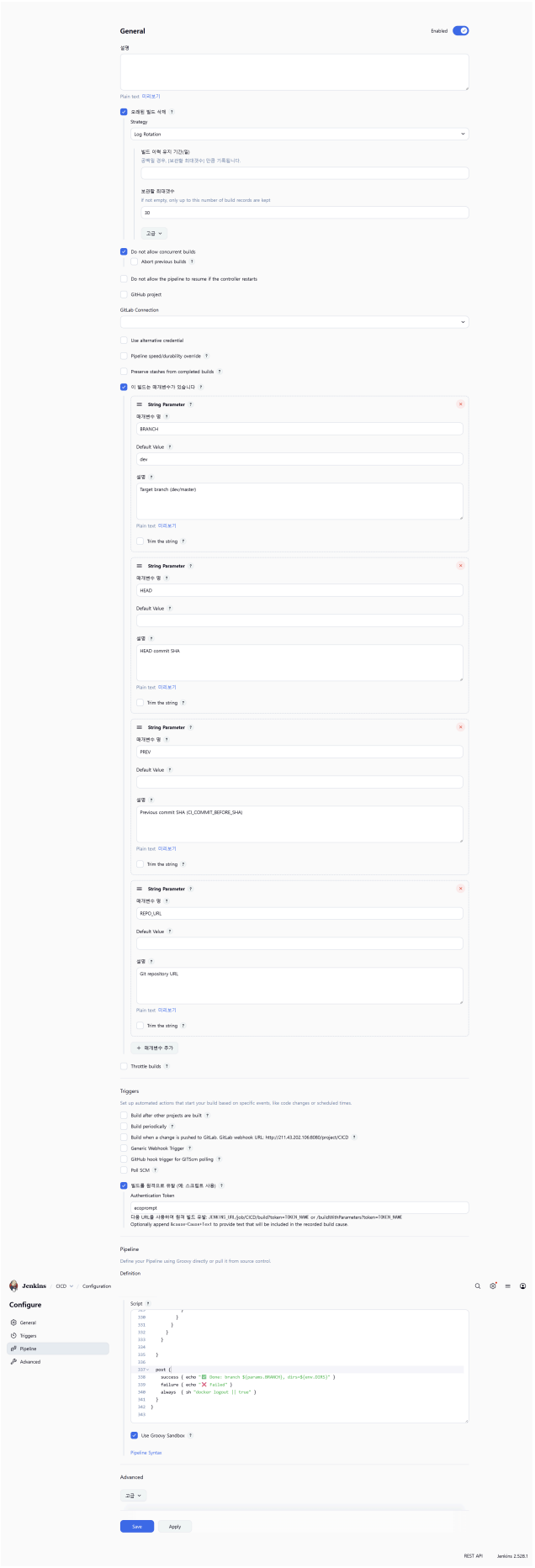
3️⃣ Jenkins 설정

✅ 3.1 Dockerfile

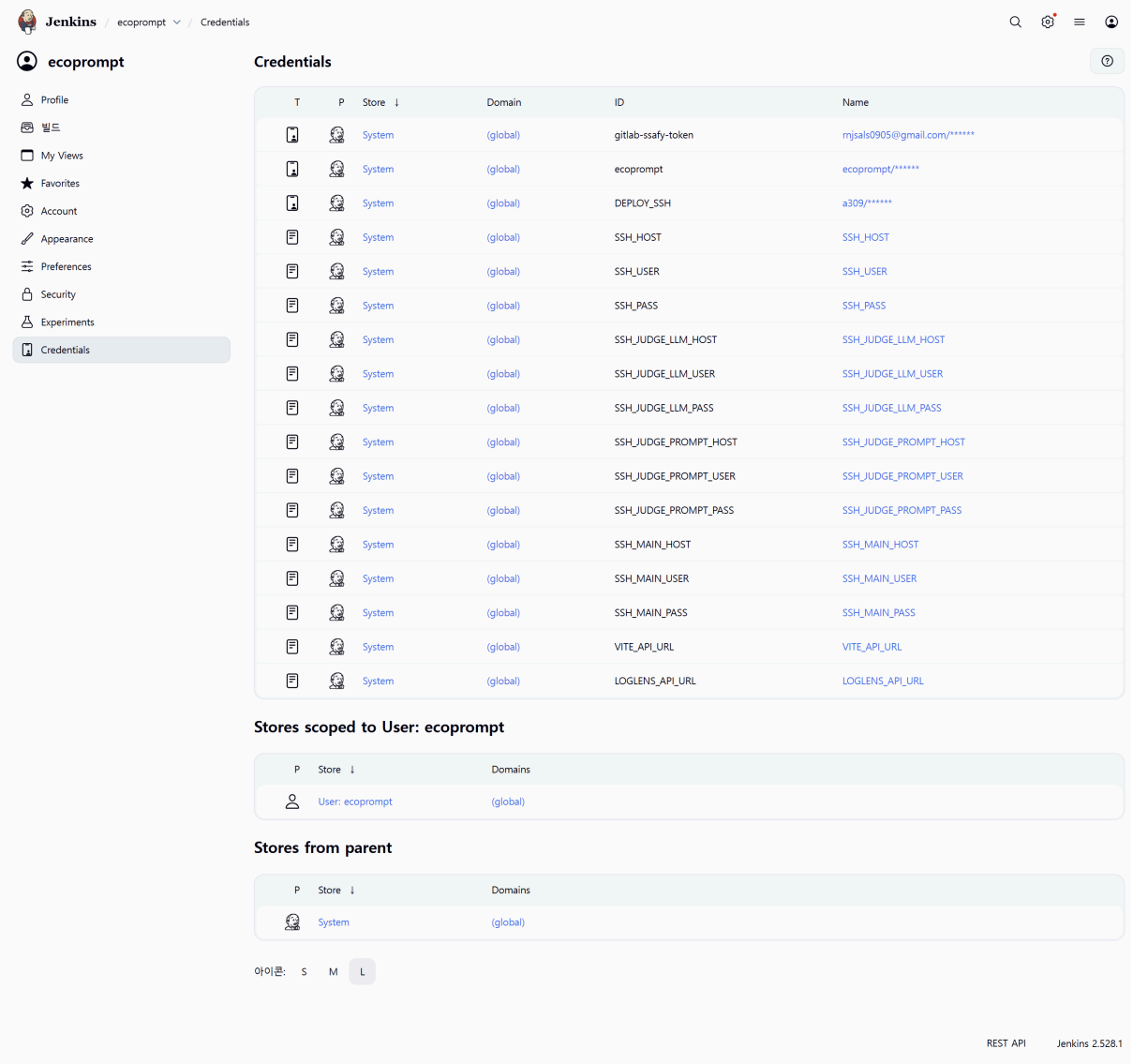
```
# Custom Jenkins image with Java 17 and Docker CLI
FROM jenkins/jenkins:lts

# Switch to root to install packages
USER root

# Install OpenJDK 17 and Docker
RUN apt-get update && \
    apt-get install -y curl docker.io openssh-client sshpass && \
    curl -fsSL https://deb.nodesource.com/setup_18.x | bash - && \
    apt-get install -y nodejs && \
    rm -rf /var/lib/apt/lists/*

# Add Jenkins user to Docker group to allow Docker commands without sudo
RUN usermod -aG docker jenkins

# Revert to Jenkins user
USER jenkins

# Ensure Jenkins HOME permissions
RUN mkdir -p /var/jenkins_home && chown -R jenkins:jenkins /var/jenkins_home

# Expose Jenkins port
EXPOSE 8080

# Default entrypoint (from base image) will start Jenkins
```

✅ 3.2 Jenkins pipeline settings

▼ 파이프라인 정보


General
Enabled
설명
Plain text 미리보기
오래된 빌드 삭제
Strategy
Log Rotation
빌드 이력 유지 기간(일)
공백일 경우, 보관할 최대갯수 만큼 기록됩니다.
보관할 최대갯수
If not empty, only up to this number of build records are kept
30
고급
Do not allow concurrent builds
Abort previous builds
Do not allow the pipeline to resume if the controller restarts
GitHub project
GitLab Connection
Use alternative credential
Pipeline speed/durability override
Preserve stashes from completed builds
이 빌드는 매개변수가 있습니다
String Parameter
매개변수 명
BRANCH
Default Value
dev
설명
Target branch (dev/master)
Plain text 미리보기
Trim the string
String Parameter
매개변수 명
HEAD
Default Value
설명
HEAD commit SHA
Plain text 미리보기
Trim the string
String Parameter
매개변수 명
PREV
Default Value
설명
Previous commit SHA (CI_COMMIT_BEFORE_SHA)
Plain text 미리보기
Trim the string
String Parameter
매개변수 명
REPO_URL
Default Value
설명
Git repository URL
Plain text 미리보기
Trim the string
매개변수 추가
Throttle builds
Triggers
Set up automated actions that start your build based on specific events, like code changes or scheduled times.
Build after other projects are built
Build periodically
Build when a change is pushed to GitLab. GitLab webhook URL: http://211.43.202.106:8080/project/CICD
Generic Webhook Trigger
GitHub hook trigger for GITScm polling
Poll SCM
빌드를 원격으로 유발 (예: 스크립트 사용)
Authentication Token
ecoprompt
다음 URL을 사용하여 원격 빌드 유발: JENKINS_URL/job/CICD/build?token=TOKEN_NAME or /buildWithParameters?token=TOKEN_NAME
Optionally append &cause=Cause+Text to provide text that will be included in the recorded build cause.
Pipeline
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Jenkins
CICD
Configuration
Configure
General
Triggers
Pipeline
Advanced
Script
post {
success { echo "✅ Done: branch ${params.BRANCH}, dirs=${env.DIRS}" }
failure { echo "❌ Failed" }
always { sh "docker logout || true" }
}
Use Groovy Sandbox
Pipeline Syntax
Advanced
고급
Save
Apply
REST API
Jenkins 2.528.1

✅ 3.3 Jenkins Credentials



✅ 3.4 Jenkins Script

```
pipeline {
  agent any

  options {
    timestamps()
    disableConcurrentBuilds()
    buildDiscarder(logRotator(numToKeepStr: '30'))
  }
```

```
  parameters {
    string(name: 'BRANCH',   defaultValue: 'dev',  description: 'Target branch (dev/mas
ter)')
    string(name: 'HEAD',     defaultValue: '',        description: 'HEAD commit SHA')
    string(name: 'PREV',     defaultValue: '',        description: 'Previous commit SHA (CI
_COMMIT_BEFORE_SHA)')
    string(name: 'REPO_URL', defaultValue: '',        description: 'Git repository URL')
  }

  environment {
    // 도커/배포 환경(예시)
    REGISTRY_URL = 'hub.docker.com'
    REGISTRY_NS  = 'ecoprompt'
    CR_CRED      = 'ecoprompt'
    DEPLOY_SSH   = 'DEPLOY_SSH'
    DOCKER_BUILDKIT = '1'
    SSH_HOST     = 'SSH_HOST'
    SSH_USER     = 'SSH_USER'
    SSH_PASS     = 'SSH_PASS'
  }

  stages {
    stage('Checkout') {
      steps {
        script {
          if (!params.REPO_URL?.trim()) {
            error "REPO_URL parameter is required."
          }
          echo "Branch=${params.BRANCH}, HEAD=${params.HEAD}, PREV=${params.
PREV}"
          // clean checkout to ensure full history availability for diff
          deleteDir()
          checkout([
            $class: 'GitSCM',
            branches: [[name: "*/${params.BRANCH}"]],
            userRemoteConfigs: [[
              url: 'https://lab.ssafy.com/s13-final/S13P31A309.git',
              credentialsId: 'gitlab-ssafy-token'
            ]],
            doGenerateSubmoduleConfigurations: false,
            extensions: [
              [$class: 'CleanCheckout'],
              [$class: 'CloneOption', depth: 0, noTags: false, reference: '', shallow: false]
```

```
          ]
        ])
      }
    }
  }

  stage('Detect changed dirs') {
    steps {
      script {
        String head = params.HEAD?.trim()
        String prev = params.PREV?.trim()

        // PREV가 비어있다면 안전한 fallback
        if (!prev) {
          prev = sh(script: "git rev-parse ${head}^ 2>/dev/null || git merge-base origin/
${params.BRANCH} ${head}", returnStdout: true).trim()
        }
        if (!head) {
          head = sh(script: "git rev-parse HEAD", returnStdout: true).trim()
        }

        echo "Diff range: ${prev}..${head}"

        def changedFilesRaw = sh(script: "git diff --name-only ${prev} ${head} || tru
e", returnStdout: true).trim()
        if (!changedFilesRaw) {
          echo "No file changes."
          env.DIRS = ''
          return
        }

        def files = changedFilesRaw.split("\\r?\\n") as List

        // 화이트리스트(프로젝트 루트 디렉토리)
        def projectDirs = ['back/', 'front/', 'llm/', 'judge_llm/', 'judge_prompt/']

        // 문서/설정만 바뀐 경우 제외하고 싶다면 ignore 목록
        def ignorePrefixes = ['README', 'docs/', '.github/', '.gitlab/', '.vscode/', '.idea/',
'Jenkinsfile', '.editorconfig', '.prettier']

        // 변경된 프로젝트 추출
        def changedDirs = [] as Set
        files.each { f →
```

```
          // 무시대상 스킵
          if (ignorePrefixes.any { ig -> f == ig || f.startsWith(ig) }) {
            return
          }
          projectDirs.each { pd ->
            if (f.startsWith(pd)) changedDirs << pd[0..-2] /* 'be/' -> 'be' */
          }
        }

        env.DIRS = changedDirs.join(',')
        echo "Changed directories: ${env.DIRS}"
        if (!env.DIRS) {
          echo "No whitelisted project changes."
          currentBuild.result = 'NOT_BUILT'
          error('Nothing to do')
        }
      }
    }
  }

  stage('Login Registry') {
    when { expression { return env.DIRS?.trim() } }
    steps {
      withCredentials([usernamePassword(credentialsId: env.CR_CRED, usernameVari
able: 'CR_USER', passwordVariable: 'CR_PASS')]) {
        sh '''
          export HOME=/var/jenkins_home
          echo "$CR_PASS" | docker login -u "$CR_USER" --password-stdin
        '''
      }
    }
  }

  /* ====== 프로젝트별 파이프라인 (변경된 디렉토리만 when으로 수행) ====== */

  stage('BE: build & push') {
    when { expression { return env.DIRS?.split(',')?.contains('back') } }
    steps {
      dir('back') {
        sh """
          echo ${REGISTRY_NS}
          docker build -t ${REGISTRY_NS}/back:${params.BRANCH} .
          docker build -t ${REGISTRY_NS}/back:latest .
```

```
        docker push ${REGISTRY_NS}/back:${params.BRANCH}
        docker push ${REGISTRY_NS}/back:latest
      """
    }
  }
}

stage('FE: build & push') {
  when { expression { return env.DIRS?.split(',')?.contains('front') } }
  steps {
    dir('front') {
      withCredentials([
        string(credentialsId: 'VITE_API_URL', variable: 'VITE_API_URL'),
        string(credentialsId: 'LOGLENS_API_URL', variable: 'LOGLENS_API_URL')]) {
        sh '''
          set -euo pipefail
          # 로그에 노출 금지: echo, cat 하지 말 것
          docker build \
            --build-arg VITE_API_URL="$VITE_API_URL" \
            --build-arg LOGLENS_API_URL="$LOGLENS_API_URL" \
            -t ${REGISTRY_NS}/front:${BRANCH} \
            -t ${REGISTRY_NS}/front:latest .

          docker push ${REGISTRY_NS}/front:${BRANCH}
          docker push ${REGISTRY_NS}/front:latest
        '''
      }
    }
  }
}


stage('LLM: build & push') {
  when { expression { return env.DIRS?.split(',')?.contains('llm') } }
  steps {
    dir('llm') {

      sh """
         echo llm
      """
        sh """
          echo ${REGISTRY_NS}
          docker build -t ${REGISTRY_NS}/llm:${params.BRANCH} .
```

```
            docker build -t ${REGISTRY_NS}/llm:latest .
            docker push ${REGISTRY_NS}/llm:${params.BRANCH}
            docker push ${REGISTRY_NS}/llm:latest
        """
    }
  }
}

stage('JUDGE_LLM: build & push') {
  when { expression { return env.DIRS?.split(',')?.contains('judge_llm') } }
  steps {
    dir('judge_llm') {

      sh """
        echo j_l
      """

      sh """
        echo ${REGISTRY_NS}
        docker build -t ${REGISTRY_NS}/judge-llm:${params.BRANCH} .
        docker build -t ${REGISTRY_NS}/judge-llm:latest .
        docker push ${REGISTRY_NS}/judge-llm:${params.BRANCH}
        docker push ${REGISTRY_NS}/judge-llm:latest
      """
    }
  }
}

stage('JUDGE_PROMPT: build & push') {
  when { expression { return env.DIRS?.split(',')?.contains('judge_prompt') } }
  steps {
    dir('judge_prompt') {
        sh """
            echo j_p
        """
      sh """
        echo ${REGISTRY_NS}
        cd jp_server
        docker build -t ${REGISTRY_NS}/judge-prompt:${params.BRANCH} .
        docker build -t ${REGISTRY_NS}/judge-prompt:latest .
        docker push ${REGISTRY_NS}/judge-prompt:${params.BRANCH}
        docker push ${REGISTRY_NS}/judge-prompt:latest
      """
```

```
      }
    }
  }

  stage('Deploy (only changed, compose, multi-host)') {
    when { expression { return env.DIRS?.trim() } }
    steps {
      script {
        echo "[DEPLOY] env.DIRS='${env.DIRS}'"

        // 1) 변경된 디렉토리
        def changed = (env.DIRS ?: '')
          .split(',')
          .collect { it.trim() }
          .findAll { it } as Set
        echo "[DEPLOY] changed dirs → ${changed}"

        // 2) 디렉토리 → compose 서비스명
        def serviceMap = [
          'back'        : 'back',
          'front'       : 'front',
          'llm'         : 'llm',
          'judge_llm'   : 'judge-llm',
          'judge_prompt' : 'judge-prompt'
        ]
        def mappedPairs = changed.collect { [dir: it, svc: serviceMap[it]] }
        echo "[DEPLOY] mapping dir→svc → ${mappedPairs}"

        def services = mappedPairs.collect { it.svc }.findAll { it } as Set
        echo "[DEPLOY] services → ${services}"
        if (!services) {
          error "[DEPLOY] No mapped compose services. Check DIRS and serviceMap
keys."
        }

        // 3) 서비스 → 호스트키
        def serviceToHostKey = [
          'back'        : 'default',
          'front'       : 'default',
          'llm'         : 'default',
          'judge-llm'   : 'judge_llm',
          'judge-prompt' : 'judge_prompt'
        ]
```

```
        // 4) 호스트 설정
        def hostConfigs = [
          'default'      : [
            hostId: 'SSH_MAIN_HOST',
            userId: 'SSH_MAIN_USER',
            passId: 'SSH_MAIN_PASS',
            remoteDir: '/home/a309/infra'
          ],
          // judge_prompt와 judge_llm 스왑했음
          'judge_llm'    : [
            hostId: 'SSH_JUDGE_LLM_HOST',
            userId: 'SSH_JUDGE_LLM_USER',
            passId: 'SSH_JUDGE_LLM_PASS',
            remoteDir: '/Users/ssafy/deploy'
          ],
          'judge_prompt' : [
            hostId: 'SSH_JUDGE_PROMPT_HOST',
            userId: 'SSH_JUDGE_PROMPT_USER',
            passId: 'SSH_JUDGE_PROMPT_PASS',
            remoteDir: '/Users/ssafy/deploy'
          ]
        ]

        // 5) 호스트별 그룹핑
        def hostServices = [:] as LinkedHashMap
        services.each { svc ->
          def key = serviceToHostKey.get(svc, 'default')
          hostServices[key] = (hostServices[key] ?: [])
          hostServices[key] << svc
        }

        echo "[DEPLOY] hostServices → ${hostServices}"
        if (!hostServices || hostServices.values().every { it.isEmpty() }) {
          error "[DEPLOY] No host resolved for services. Check serviceToHostKey key
s."
        }

        // 6) 실제 배포
        hostServices.each { hostKey, svcList ->
          if (!svcList) return
          def cfg = hostConfigs[hostKey]
          if (!cfg) error "[DEPLOY] Missing hostConfigs for '${hostKey}'"
```

```
        def serviceList = svcList.unique().join(' ')
        echo "[DEPLOY] Host='${hostKey}', Services='${serviceList}', RemoteDir
='${cfg.remoteDir}'"

        // judge_llm일 때만 SSH 포트 22 사용
        def sshPortOpt = (hostKey == 'judge_llm') ? '-p 18080' : ''

        withCredentials([
          string(credentialsId: cfg.hostId, variable: 'SSH_HOST'),
          string(credentialsId: cfg.userId, variable: 'SSH_USER'),
          string(credentialsId: cfg.passId, variable: 'SSH_PASS')
        ]) {
          sh """
            set -euo pipefail
            echo "[DEPLOY:${hostKey}] Target services: ${serviceList}"
            sshpass -p "\$SSH_PASS" ssh ${sshPortOpt} -o StrictHostKeyChecking=n
o "\$SSH_USER@\$SSH_HOST" '
              set -euo pipefail
              # macOS PATH 보강
              if [ "\$(uname)" = "Darwin" ]; then
                 export PATH="\$PATH:/usr/local/bin:/opt/homebrew/bin:/Applications/D
ocker.app/Contents/Resources/bin"
              fi

              cd ${cfg.remoteDir}
              docker compose pull ${serviceList}
              docker compose up -d ${serviceList}
              docker compose ps
            '
          """
        }
       }
      }
    }
  }

 }

 post {
  success { echo "✅ Done: branch ${params.BRANCH}, dirs=${env.DIRS}" }
  failure { echo "❌ Failed" }
  always  { sh "docker logout || true" }
```

```
  }
}
```

4️⃣ 개발 환경

✅ 4.1 FrontEnd

✔️ 4.1.1 주요 프레임워크 및 라이브러리

- React : `19.2.0`
- React DOM : `19.2.0`
- React Router : `7.9.4` (react-router-dom 기준)
- Zustand : `5.0.8`
- React Markdown : `10.1.0` + remark-gfm / remark-math / rehype-raw / rehype-katex
- React Syntax Highlighter : `16.1.0`
- React KaTeX : `3.1.0`

✔️ 4.1.2 데이터 통신 및 실시간

- Axios : `1.13.2`

✔️ 4.1.3 개발 도구 및 환경 설정

- TypeScript : `5.9.3`
- **Vite :** `7.1.7`
- ESLint : `9.38.0`
- Prettier : `3.6.2`
- @typescript-eslint/eslint-plugin / parser : `8.46.2`
- PWA 지원 : vite-plugin-pwa 1.1.0

✅ 4.2 BackEnd

✔️ 4.2.1 개발 환경

- IDE : IntelliJ IDEA

✔️ 4.2.2 언어 및 핵심 프레임워크

- Java : **17 (OpenJDK / Liberica)**
  - Dockerfile 기준 `bellsoft/liberica-openjdk-debian:17`, `bellsoft/liberica-openjre-alpine:17` 사용 2fafbcb3-5d62-4e82-bd31-3a9f58c...
- Spring Boot : **3.5.6**
- Spring Framework :**3.5.6 BOM 기반**
- Spring Security : **3.5.6 BOM 기반**
- Spring Web / WebFlux :
  - Spring Web (REST API) : `spring-boot-starter-web`
  - Spring WebFlux (SSE 스트리밍) : `spring-boot-starter-webflux`
  - **Spring WebSocket : 별도 starter 미사용 (SSE 기반 실시간 통신)**

✔️ 4.2.3 데이터베이스 및 데이터 처리

- Database : MySQL
- MySQL Connector/J : `9.2.0`
- Spring Data JPA : `3.5.1`
- Hibernate ORM : 6.6.18.Final
- Spring Data MongoDB : `4.5.1`
- Spring Data Redis : `3.5.1`
- Lettuce (Redis Client) : 6.6.0.RELEASE

✔️ 4.2.4 API 문서화

- SpringDoc OpenAPI : **2.6.0**
- Swagger UI : **springdoc-openapi-starter-webmvc-ui**

✅ 4.3 AI

✔️ 4.3.1 언어 및 프레임워크

- Python : `3.12.3`
- FastAPI :
  - Judge LLM : `0.119.1`
  - Judge Prompt : `0.119.1`

    - LLM 서비스 : **0.119.0**
- ASGI 서버 : **Uvicorn 0.38.0**

✔️ 4.3.2 주요 라이브러리

1) 공통 인프라/유틸

- Pydantic 2.12.x, pydantic-core 2.41.x
- python-dotenv 1.1.x
- httpx 0.28.x, requests 2.32.5, urllib3 2.5.0, idna 3.11, charset-normalizer 3.4.4
- AnyIO 4.11.0, h11 0.16.0, uvloop 0.22.1, watchfiles 1.1.1

2) Judge LLM (API 서버)

- Presidio Analyzer / Anonymizer : 2.2.355
- spaCy : 3.7.4
- langdetect : 1.0.9
- pymongo : 4.15.3 (MongoDB 연동)
- rich, prometheus_client (로깅/모니터링)

3) Judge Prompt

- llama_cpp_python : 0.3.16 (로컬 GGUF LLM 연동)
- huggingface-hub : 0.35.3
- nltk : 3.9.2 (+ stopwords, punkt 다운로드)
- kiwipiepy / kiwipiepy_model : 0.21.0 (한국어 형태소 분석)
- diskcache : 5.6.3 (캐시)
- APScheduler : 3.11.1 (스케줄링)

4) LLM 서버

- sentence-transformers : 5.1.2
- LangChain 패밀리
    - langchain, langchain-community, langchain-core, langchain-huggingface, langchain-mongodb
- vLLM : 0.10.2 (고성능 LLM 서빙)
- 학습/튜닝 관련
    - wandb, accelerate, peft, trl, bitsandbytes
- 스토리지/백엔드
    - pymongo, dnspython, boto3

- PDF 생성
  - reportlab

5) OCR 서비스

- PyMuPDF : 1.26.6 (PDF 렌더링)
- pytesseract : 0.3.13 (Tesseract OCR)
- Pillow : 12.0.0 (이미지 처리)
- boto3 / botocore / s3transfer (AWS S3 연동)
- pymongo, python-dateutil, python-dotenv 등

레포지토리 구조

▼ tree S13P31A309

```
S13P31A309/
├── back
│   ├── build.gradle
│   ├── Dockerfile
│   ├── gradle
│   │   └── wrapper
│   │       ├── gradle-wrapper.jar
│   │       └── gradle-wrapper.properties
│   ├── gradlew
│   ├── gradlew.bat
│   ├── settings.gradle
│   └── src
│       ├── main
│       │   ├── java
│       │   │   └── com
│       │   │       └── closeai
│       │   │           └── ecoprompt
│       │   │               ├── ai
│       │   │               │   ├── model
│       │   │               │   │   ├── dto
│       │   │               │   │   │   ├── request
│       │   │               │   │   │   │   ├── InputJudgeRequest.java
│       │   │               │   │   │   │   └── LlmRequest.java
│       │   │               │   │   │   └── response
│       │   │               │   │   │       ├── InputJudgeResponse.java
│       │   │               │   │   │       └── LlmResponse.java
│       │   │               │   │   └── event
│       │   │               │   │       ├── EachModelEvent.java
```

```
│   │   │   │   │   ├── JudgeModelCompleteEvent.java
│   │   │   │   │   ├── LlmModelCompleteEvent.java
│   │   │   │   │   ├── ModelCancelledEvent.java
│   │   │   │   │   ├── ModelErrorEvent.java
│   │   │   │   │   └── ScoreInfo.java
│   │   │   │   └── service
│   │   │   │       └── AiService.java
│   │   │   ├── bookmark
│   │   │   │   ├── controller
│   │   │   │   │   ├── BookmarkControllerDocs.java
│   │   │   │   │   └── BookmarkController.java
│   │   │   │   ├── model
│   │   │   │   │   ├── dto
│   │   │   │   │   │   ├── request
│   │   │   │   │   │   │   ├── CreateBookmarkRequest.java
│   │   │   │   │   │   │   ├── DeleteBookmarkRequest.java
│   │   │   │   │   │   │   └── UpdateBookmarkSequence.java
│   │   │   │   │   │   └── response
│   │   │   │   │   │       ├── BookmarkResponse.java
│   │   │   │   │   │       ├── CreateBookmarkResponse.java
│   │   │   │   │   │       └── GetBookmarkResponse.java
│   │   │   │   │   └── entity
│   │   │   │   │       └── Bookmark.java
│   │   │   │   ├── repository
│   │   │   │   │   └── BookmarkRepository.java
│   │   │   │   └── service
│   │   │   │       └── BookmarkService.java
│   │   │   ├── chatting
│   │   │   │   ├── controller
│   │   │   │   │   ├── ChattingControllerDocs.java
│   │   │   │   │   └── ChattingController.java
│   │   │   │   ├── model
│   │   │   │   │   ├── dto
│   │   │   │   │   │   ├── request
│   │   │   │   │   │   │   ├── UpdateChattingProjectRequest.java
│   │   │   │   │   │   │   └── UpdateChattingTitleRequest.java
│   │   │   │   │   │   └── response
│   │   │   │   │   │       ├── ChattingResponse.java
│   │   │   │   │   │       └── ChattingWithLastMessageResponse.java
│   │   │   │   │   └── entity
│   │   │   │   │       └── Chatting.java
│   │   │   │   ├── repository
│   │   │   │   │   └── ChattingRepository.java
```

```
│   │   │       │   └── service
│   │   │       │       └── ChattingService.java
│   │   │       ├── common
│   │   │       │   ├── ApiResponse.java
│   │   │       │   ├── config
│   │   │       │   │   ├── AsyncConfig.java
│   │   │       │   │   ├── JpaConfig.java
│   │   │       │   │   ├── MongoConfig.java
│   │   │       │   │   ├── RedisCacheConfig.java
│   │   │       │   │   ├── RedisConfig.java
│   │   │       │   │   ├── SchedulingConfig.java
│   │   │       │   │   ├── SecurityConfig.java
│   │   │       │   │   ├── SshDataSourceConfig.java
│   │   │       │   │   ├── SshTunnelingInitializer.java
│   │   │       │   │   ├── SwaggerConfig.java
│   │   │       │   │   └── WebClientConfig.java
│   │   │       │   ├── CustomUtil.java
│   │   │       │   ├── entity
│   │   │       │   │   ├── BaseEntity.java
│   │   │       │   │   └── MongoBaseEntity.java
│   │   │       │   ├── exception
│   │   │       │   │   ├── BusinessException.java
│   │   │       │   │   ├── ErrorData.java
│   │   │       │   │   └── GlobalExceptionHandler.java
│   │   │       │   ├── logging
│   │   │       │   │   ├── AppLogger.java
│   │   │       │   │   └── Logging.md
│   │   │       │   └── service
│   │   │       │       └── CacheTimeService.java
│   │   │       ├── dashboard
│   │   │       │   ├── controller
│   │   │       │   │   ├── DashboardControllerDocs.java
│   │   │       │   │   └── DashboardController.java
│   │   │       │   ├── model
│   │   │       │   │   └── dto
│   │   │       │   │       └── response
│   │   │       │   │           ├── DashboardDetailScoreResponse.java
│   │   │       │   │           ├── DetailScoreResponse.java
│   │   │       │   │           ├── EcoPickResponse.java
│   │   │       │   │           └── PersonalStateResponse.java
│   │   │       │   └── service
│   │   │       │       └── DashboardService.java
│   │   │       ├── EcoPromptApplication.java
```

```
│   │   │       ├── message
│   │   │       │   ├── controller
│   │   │       │   │   ├── MessageControllerDocs.java
│   │   │       │   │   └── MessageController.java
│   │   │       │   ├── model
│   │   │       │   │   ├── dto
│   │   │       │   │   │   ├── request
│   │   │       │   │   │   │   ├── SubmitMessageRequest.java
│   │   │       │   │   │   │   └── UpdateMessageRequest.java
│   │   │       │   │   │   └── response
│   │   │       │   │   │       ├── AiMessage.java
│   │   │       │   │   │       ├── DailyRankingProjection.java
│   │   │       │   │   │       ├── EcoPickFlatProjection.java
│   │   │       │   │   │       ├── GetMessageResponse.java
│   │   │       │   │   │       ├── GetScoreInfo.java
│   │   │       │   │   │       ├── JudgeOnlyResponse.java
│   │   │       │   │   │       ├── MessageKeywordDto.java
│   │   │       │   │   │       ├── MessagePageResponse.java
│   │   │       │   │   │       ├── ScoreMessage.java
│   │   │       │   │   │       ├── SearchMessageResponse.java
│   │   │       │   │   │       ├── SubmitMessageResponse.java
│   │   │       │   │   │       └── UserMessage.java
│   │   │       │   │   └── entity
│   │   │       │   │       ├── MessageDocument.java
│   │   │       │   │       ├── Message.java
│   │   │       │   │       ├── MessageSender.java
│   │   │       │   │       └── MessageStatus.java
│   │   │       │   ├── repository
│   │   │       │   │   ├── MessageJpaRepository.java
│   │   │       │   │   └── mongo
│   │   │       │   │       └── MessageMongoRepository.java
│   │   │       │   └── service
│   │   │       │       ├── MessageEventHandler.java
│   │   │       │       ├── MessageService.java
│   │   │       │       └── SseService.java
│   │   │       ├── mileage
│   │   │       │   ├── model
│   │   │       │   │   └── entity
│   │   │       │   │       └── Mileage.java
│   │   │       │   ├── repository
│   │   │       │   │   └── MileageRepository.java
│   │   │       │   └── service
│   │   │       │       └── MileageService.java
```

```
│   │   │       ├── mr
│   │   │       │   ├── controller
│   │   │       │   │   ├── GitlabWebhookController.java
│   │   │       │   │   └── MrAutomationController.java
│   │   │       │   ├── model
│   │   │       │   │   └── dto
│   │   │       │   │       ├── request
│   │   │       │   │       │   ├── AiMrAnalyzeRequest.java
│   │   │       │   │       │   ├── GitlabMergeRequestEvent.java
│   │   │       │   │       │   └── MrGeneratorRequest.java
│   │   │       │   │       └── response
│   │   │       │   │           ├── AiMrAnalyzeResponse.java
│   │   │       │   │           ├── GitlabMrChangesResponse.java
│   │   │       │   │           └── MrGeneratorResponse.java
│   │   │       │   └── service
│   │   │       │       ├── AiClient.java
│   │   │       │       ├── DiffUtils.java
│   │   │       │       ├── GitlabApiClient.java
│   │   │       │       ├── MergeRequestService.java
│   │   │       │       └── MrPromptBuilder.java
│   │   │       ├── project
│   │   │       │   ├── controller
│   │   │       │   │   ├── ProjectControllerDocs.java
│   │   │       │   │   └── ProjectController.java
│   │   │       │   ├── model
│   │   │       │   │   ├── dto
│   │   │       │   │   │   ├── request
│   │   │       │   │   │   │   ├── PersonalProjectRequest.java
│   │   │       │   │   │   │   └── ProjectUpdateRequest.java
│   │   │       │   │   │   └── response
│   │   │       │   │   │       ├── CreateProjectResponse.java
│   │   │       │   │   │       ├── PersonalProjectResponse.java
│   │   │       │   │   │       ├── SidebarResponse.java
│   │   │       │   │   │       └── SpecificProjectResponse.java
│   │   │       │   │   └── entity
│   │   │       │   │       └── Project.java
│   │   │       │   ├── repository
│   │   │       │   │   └── ProjectRepository.java
│   │   │       │   └── service
│   │   │       │       └── ProjectService.java
│   │   │       ├── ranking
│   │   │       │   ├── controller
│   │   │       │   │   ├── RankingControllerDocs.java
```

```
│   │   │   │   │   └── RankingController.java
│   │   │   │   ├── model
│   │   │   │   │   ├── dto
│   │   │   │   │   │   └── response
│   │   │   │   │   │       ├── RankingResponse.java
│   │   │   │   │   │       └── TodayRankingResponse.java
│   │   │   │   │   └── entity
│   │   │   │   │       ├── RankingChange.java
│   │   │   │   │       └── Ranking.java
│   │   │   │   ├── repository
│   │   │   │   │   └── RankingRepository.java
│   │   │   │   └── service
│   │   │   │       ├── RankingService.java
│   │   │   │       └── RankingSnapshotJob.java
│   │   │   ├── score
│   │   │   │   ├── model
│   │   │   │   │   └── entity
│   │   │   │   │       └── Score.java
│   │   │   │   ├── repository
│   │   │   │   │   ├── redis
│   │   │   │   │   └── ScoreRepository.java
│   │   │   │   └── service
│   │   │   │       └── ScoreService.java
│   │   │   ├── security
│   │   │   │   ├── cookie
│   │   │   │   │   └── TokenCookieManager.java
│   │   │   │   ├── filter
│   │   │   │   │   ├── JwtAuthenticationFilter.java
│   │   │   │   │   └── RefreshFilter.java
│   │   │   │   ├── jwt
│   │   │   │   │   ├── dto
│   │   │   │   │   │   └── response
│   │   │   │   │   │       └── IssueResponse.java
│   │   │   │   │   └── JwtUtil.java
│   │   │   │   ├── logout
│   │   │   │   │   ├── CookieLogoutHandler.java
│   │   │   │   │   └── SimpleLogoutSuccessHandler.java
│   │   │   │   └── oauth
│   │   │   │       ├── OAuth2SuccessHandler.java
│   │   │   │       └── SsafyOAuth2UserService.java
│   │   │   ├── training
│   │   │   │   ├── model
│   │   │   │   │   └── dto
```

```
│   │   │   │   │   ├── request
│   │   │   │   │   │   └── TrainingRequest.java
│   │   │   │   │   └── TrainingItem.java
│   │   │   │   └── service
│   │   │   │       └── TrainingTriggerJob.java
│   │   │   ├── user
│   │   │   │   ├── controller
│   │   │   │   │   ├── AuthControllerDocs.java
│   │   │   │   │   └── AuthController.java
│   │   │   │   ├── model
│   │   │   │   │   └── entity
│   │   │   │   │       ├── RefreshSession.java
│   │   │   │   │       └── User.java
│   │   │   │   ├── repository
│   │   │   │   │   ├── redis
│   │   │   │   │   │   └── RefreshSessionRepository.java
│   │   │   │   │   └── UserRepository.java
│   │   │   │   └── service
│   │   │   │       └── TokenService.java
│   │   │   └── userinfo
│   │   │       ├── controller
│   │   │       │   ├── UserInfoControllerDocs.java
│   │   │       │   └── UserInfoController.java
│   │   │       ├── model
│   │   │       │   ├── dto
│   │   │       │   │   ├── request
│   │   │       │   │   │   └── UpdatePersonalPromptRequest.java
│   │   │       │   │   └── response
│   │   │       │   │       ├── GetUserInfoResponse.java
│   │   │       │   │       └── SharingInformationStatusResponse.java
│   │   │       │   └── entity
│   │   │       │       └── UserInfo.java
│   │   │       ├── repository
│   │   │       │   └── UserInfoRepository.java
│   │   │       └── service
│   │   │           └── UserInfoService.java
│   │   └── resources
│   │       └── application.yml
│   └── test
│       └── java
│           └── com
│               └── closeai
│                   └── ecoprompt
```

```
│                   └── EcoPromptApplicationTests.java
├── front
│   ├── Dockerfile
│   ├── eslint.config.js
│   ├── index.html
│   ├── package.json
│   ├── package-lock.json
│   ├── postcss.config.cjs
│   ├── public
│   │   ├── fonts
│   │   │   └── PretendardVariable.woff2
│   │   ├── icons
│   │   │   ├── 1.svg
│   │   │   ├── 2.svg
│   │   │   ├── 3.svg
│   │   │   ├── add_chat.svg
│   │   │   ├── add_folder.svg
│   │   │   ├── add.svg
│   │   │   ├── back.svg
│   │   │   ├── chat.svg
│   │   │   ├── chevron.svg
│   │   │   ├── close.svg
│   │   │   ├── copy.svg
│   │   │   ├── dashboard.svg
│   │   │   ├── delete.svg
│   │   │   ├── down.svg
│   │   │   ├── edit.svg
│   │   │   ├── error.svg
│   │   │   ├── find.svg
│   │   │   ├── folder_open.svg
│   │   │   ├── folder.svg
│   │   │   ├── forward.svg
│   │   │   ├── help.svg
│   │   │   ├── info.svg
│   │   │   ├── keep.svg
│   │   │   ├── menu.svg
│   │   │   ├── more_detail.svg
│   │   │   ├── mr_create.svg
│   │   │   ├── new.svg
│   │   │   ├── profile.svg
│   │   │   ├── prompt.svg
│   │   │   ├── rank.svg
│   │   │   ├── search.svg
```

```
│   │   │   ├── send.svg
│   │   │   ├── settings.svg
│   │   │   ├── sidebar_close.svg
│   │   │   ├── sidebar_open.svg
│   │   │   ├── stop.svg
│   │   │   └── up.svg
│   │   └── logo
│   │       ├── ggb_logo_name.png
│   │       ├── ggb_logo_png_name.png
│   │       ├── ggb_logo_png.png
│   │       ├── header_img.png
│   │       ├── ngb_logo_name.png
│   │       ├── ngb_logo_png_name.png
│   │       ├── ngb_logo_png.png
│   │       ├── wgb_logo_name.png
│   │       ├── wgb_logo_png_name.png
│   │       └── wgb_logo_png.png
│   ├── src
│   │   ├── components
│   │   │   ├── auth
│   │   │   │   └── ConsentGuard.tsx
│   │   │   ├── chat
│   │   │   │   ├── AIMessage.tsx
│   │   │   │   ├── ChatInput.tsx
│   │   │   │   ├── ChatLoading.tsx
│   │   │   │   ├── ErrorMessage.tsx
│   │   │   │   ├── MessageList.tsx
│   │   │   │   ├── PromptScore.tsx
│   │   │   │   └── UserMessage.tsx
│   │   │   ├── common
│   │   │   │   ├── Bottombar.tsx
│   │   │   │   ├── Button.tsx
│   │   │   │   ├── MenuItem.tsx
│   │   │   │   ├── Sheet.tsx
│   │   │   │   ├── Sidebar
│   │   │   │   │   ├── ChatListItem.tsx
│   │   │   │   │   ├── ChatList.tsx
│   │   │   │   │   ├── ChatMenu.tsx
│   │   │   │   │   ├── ContextMenu.tsx
│   │   │   │   │   ├── index.ts
│   │   │   │   │   ├── MenuItem.tsx
│   │   │   │   │   ├── ProjectListItem.tsx
│   │   │   │   │   ├── ProjectList.tsx
```

```
│   │   │   │   │   ├── ProjectMenu.tsx
│   │   │   │   │   ├── SidebarCollapsed.tsx
│   │   │   │   │   ├── SidebarFooter.tsx
│   │   │   │   │   ├── SidebarHeader.tsx
│   │   │   │   │   └── Sidebar.tsx
│   │   │   │   ├── TextArea.tsx
│   │   │   │   ├── TextField.tsx
│   │   │   │   ├── Toast.tsx
│   │   │   │   ├── Toggle.tsx
│   │   │   │   ├── Tooltip.tsx
│   │   │   │   └── Topbar.tsx
│   │   │   ├── dashboard
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   ├── EcoPick.tsx
│   │   │   │   └── Ranking.tsx
│   │   │   ├── home
│   │   │   │   └── MainChat.tsx
│   │   │   ├── project
│   │   │   │   ├── ChatCard.tsx
│   │   │   │   ├── ProjectOverlay.tsx
│   │   │   │   └── ProjectOverview.tsx
│   │   │   ├── project_create
│   │   │   │   ├── ProjectCreateForm.tsx
│   │   │   │   └── ProjectCreateOverlay.tsx
│   │   │   ├── search
│   │   │   │   └── SearchModal.tsx
│   │   │   └── settings
│   │   │       ├── SettingsForm.tsx
│   │   │       └── SettingsOverlay.tsx
│   │   ├── constants
│   │   │   └── ui.ts
│   │   ├── context
│   │   │   ├── AppShellContext.tsx
│   │   │   └── ToastContext.tsx
│   │   ├── counter.ts
│   │   ├── data
│   │   │   └── mockData.ts
│   │   ├── hooks
│   │   │   ├── useChatScroll.ts
│   │   │   ├── useClickOutside.ts
│   │   │   ├── useContextMenu.ts
│   │   │   ├── useDeviceMode.ts
│   │   │   ├── useInfiniteScroll.ts
```

```
│   │   │   ├── useLongPress.ts
│   │   │   └── useSidebarData.ts
│   │   ├── layouts
│   │   │   └── AppShell.tsx
│   │   ├── lib
│   │   │   ├── deviceMode.ts
│   │   │   └── viewport.ts
│   │   ├── main.tsx
│   │   ├── pages
│   │   │   ├── Chat.tsx
│   │   │   ├── Consent.tsx
│   │   │   ├── Dashboard.tsx
│   │   │   ├── Home.tsx
│   │   │   ├── Landing.tsx
│   │   │   ├── MRGenerator.tsx
│   │   │   ├── Project.tsx
│   │   │   └── Settings.tsx
│   │   ├── services
│   │   │   ├── api
│   │   │   │   ├── auth.ts
│   │   │   │   ├── chatting.ts
│   │   │   │   ├── dashboard.ts
│   │   │   │   ├── message.ts
│   │   │   │   ├── mr-generator.ts
│   │   │   │   ├── project.ts
│   │   │   │   ├── ranking.ts
│   │   │   │   └── user-info.ts
│   │   │   └── axios.ts
│   │   ├── store
│   │   │   ├── chatStore.ts
│   │   │   └── projectStore.ts
│   │   ├── styles
│   │   │   ├── app.css
│   │   │   ├── components
│   │   │   │   ├── chat
│   │   │   │   │   ├── ai-message.css
│   │   │   │   │   ├── chat-input.css
│   │   │   │   │   ├── chat-loading.css
│   │   │   │   │   ├── error-message.css
│   │   │   │   │   ├── prompt-score.css
│   │   │   │   │   └── user-message.css
│   │   │   │   ├── common
│   │   │   │   │   ├── bottombar.css
```

```
│   │   │   │   ├── button.css
│   │   │   │   ├── sheet.css
│   │   │   │   ├── sidebar
│   │   │   │   │   └── index.css
│   │   │   │   ├── text-area.css
│   │   │   │   ├── text-field.css
│   │   │   │   ├── toast.css
│   │   │   │   ├── toggle.css
│   │   │   │   ├── tooltip.css
│   │   │   │   └── topbar.css
│   │   │   ├── dashboard
│   │   │   │   ├── dashboard.css
│   │   │   │   ├── eco-pick.css
│   │   │   │   └── ranking.css
│   │   │   ├── home
│   │   │   │   └── main-chat.css
│   │   │   ├── project
│   │   │   │   └── project-create.css
│   │   │   ├── search
│   │   │   │   └── search-modal.css
│   │   │   └── settings
│   │   │       └── settings-form.css
│   │   └── pages
│   │       ├── chat.css
│   │       ├── consent.css
│   │       ├── dashboard.css
│   │       ├── home.css
│   │       ├── landing.css
│   │       ├── mr-generator.css
│   │       ├── project.css
│   │       └── settings.css
│   ├── types
│   │   ├── api
│   │   │   ├── auth.types.ts
│   │   │   ├── chatting.types.ts
│   │   │   ├── dashboard.types.ts
│   │   │   ├── message.types.ts
│   │   │   ├── mr-generator.types.ts
│   │   │   ├── project.types.ts
│   │   │   ├── ranking.types.ts
│   │   │   └── user-info.types.ts
│   │   ├── chat.types.ts
│   │   ├── navigation.types.ts
```

```
│   │   │   └── sidebar.types.ts
│   │   ├── typescript.svg
│   │   └── utils
│   │       ├── date.ts
│   │       ├── id.ts
│   │       ├── messageParser.ts
│   │       └── sseListeners.ts
│   ├── tsconfig.json
│   └── vite.config.ts
├── judge_llm
│   ├── api
│   │   ├── app
│   │   │   ├── adapters
│   │   │   │   ├── db
│   │   │   │   │   ├── __init__.py
│   │   │   │   │   ├── mongo_connector.py
│   │   │   │   │   └── train_repository.py
│   │   │   │   ├── __init__.py
│   │   │   │   ├── judge_llm_http.py
│   │   │   │   └── main_llm_http.py
│   │   │   ├── dummies.py
│   │   │   ├── __init__.py
│   │   │   ├── masking
│   │   │   │   ├── high_entropy_recognizer.py
│   │   │   │   ├── __init__.py
│   │   │   │   ├── kr_address_recognizer.py
│   │   │   │   ├── kr_bank_recognizer.py
│   │   │   │   ├── kr_bizno_recognizer.py
│   │   │   │   ├── kr_email_recognizer.py
│   │   │   │   ├── kr_ip_recognizer.py
│   │   │   │   ├── kr_secret_ext_recognizer.py
│   │   │   │   ├── presidio_adapter.py
│   │   │   │   └── regex_rules.py
│   │   │   ├── models.py
│   │   │   ├── pipeline.py
│   │   │   ├── ports.py
│   │   │   └── wiring.py
│   │   ├── __init__.py
│   │   └── requirements.txt
│   ├── Dockerfile
│   ├── main.py
│   ├── model
│   │   ├── eval
```

```
│   │   │   ├── build_kosimpleeval_batch.py
│   │   │   ├── eval_kosimple.py
│   │   │   ├── judge_eval.py
│   │   │   ├── judge_prompts.py
│   │   │   └── kosimpleeval_KMMLU_test.csv
│   │   ├── requirements_model.txt
│   │   └── scripts
│   │       ├── healthcheck.sh
│   │       ├── start_server.sh
│   │       └── watchdog.sh
│   └── tests
│       ├── conftest.py
│       ├── generate_samples.py
│       ├── samples.jsonl
│       ├── test_judge_api.py
│       ├── test_presidio_ko.py
│       └── test_regex_rules.py
├── judge_prompt
│   ├── benchmark
│   │   ├── append_source.py
│   │   ├── awsome_prompt.py
│   │   ├── dataset_frame.py
│   │   ├── debug_conv.py
│   │   ├── extract.py
│   │   ├── gpt_conv.py
│   │   ├── last_conv.py
│   │   ├── preprocessed
│   │   │   ├── c_extract.py
│   │   │   ├── colon_extract.py
│   │   │   └── split_dataset.py
│   │   ├── requirements_bm.txt
│   │   └── stackoverflow_dataset.py
│   ├── jp_server
│   │   ├── app
│   │   │   ├── api
│   │   │   │   └── inference.py
│   │   │   ├── core
│   │   │   │   └── config.py
│   │   │   ├── __init__.py
│   │   │   ├── main.py
│   │   │   ├── schemas
│   │   │   │   ├── request.py
│   │   │   │   └── response.py
```

```
│   │   ├── services
│   │   │   ├── crawler
│   │   │   │   └── live_crawler.py
│   │   │   ├── infra
│   │   │   │   └── worker_pool.py
│   │   │   ├── __init__.py
│   │   │   └── judge
│   │   │       ├── __init__.py
│   │   │       ├── judge_service.py
│   │   │       ├── llama_client.py
│   │   │       └── tokenizer_config.py
│   │   └── tests
│   │       ├── __init__.py
│   │       ├── test_split_sentences.py
│   │       └── test_tokenizer_features.py
│   ├── Dockerfile
│   └── requirements_JP.txt
├── models
│   ├── qwen2.5-3b
│   └── qwen2.5-7b
└── model_server
    ├── app
    │   ├── api
    │   │   └── router.py
    │   ├── core
    │   │   └── config.py
    │   ├── main.py
    │   ├── schemas
    │   │   ├── request.py
    │   │   └── response.py
    │   └── services
    │       ├── json_grammar.py
    │       ├── judge_service.py
    │       └── model_loader.py
    └── requirements_JP.txt
└── llm
    ├── app
    │   ├── api
    │   │   ├── __init__.py
    │   │   └── v1
    │   │       ├── endpoints
    │   │       │   ├── __init__.py
    │   │       │   ├── chat.py
```

```
│   │   │   └── train.py
│   │   ├── __init__.py
│   │   └── routers.py
│   ├── core
│   │   ├── __init__.py
│   │   ├── config.py
│   │   └── concurrency.py
│   ├── models
│   │   ├── __init__.py
│   │   ├── llm_loader.py
│   │   ├── mongodb_loader.py
│   │   ├── prompt_template.py
│   │   └── vectordb_loader.py
│   ├── schemas
│   │   ├── __init__.py
│   │   ├── chat.py
│   │   └── train.py
│   └── services
│       ├── __init__.py
│       ├── dpo_train.py
│       ├── evaluate.py
│       ├── chat.py
│       ├── load_dpo_datasets.py
│       ├── model_download.py
│       ├── pdf_style.py
│       ├── pdf_tools.py
│       ├── routing.py
│       └── use_mongodb.py
├── Dockerfile
├── main.py
├── pyproject.toml
├── README.md
├── requirements.txt
├── training
│   ├── load_training_datasets.py
│   └── sft_lora_fine_tuning.py
└── uv.lock
```